


# 1 - Introduction

Hadrien Grasland

IJCLab – Orsay

19/10/2020

# Tour de table

# Premier survol de Git

- **Fonction technique :**
  - Gère des versions d'une arborescence de fichiers
  - Plutôt orienté texte brut (ex : code, configuration…)
  - Décentralisé (synchronisation sans serveur maître)
  - Supporte les divergences et manipulations d'historique
- **Importance politique :**
  - Moteur de forges sociales, très utilisées en open-source
  - Présent aussi chez Google, Microsoft, Netflix, le CERN…

# Gestionnaires de versions

- **Alternative à la maintenance manuelle de copies**
  - Gourmande en espace disque
  - Organisation non standardisée
  - Comparaisons entre versions difficiles
- **Trois générations historiques :**
  - 1970+ : Versions d'un fichier local unique (SCCS, RCS…)
  - 1980+ : Répertoires & modèle client-serveur (CVS, SVN…)
  - 2000+ : Atomique & décentralisé (BitKeeper, Git, Hg…)

# Que versionne-t-on ?

- **Un fichier, ou toute une aborescence ?**
  - Fichier : Moins coûteux pour la machine
  - Arborescence : Plus pertinent pour l'utilisateur
- **Pourquoi privilégier le texte brut ?**
  - Facile de calculer des différences « minimales »
    → Gain de place, moins de conflits de fusion…
  - Un cadre encore très large : codes sources, fichiers de configuration, édition littéraire collaborative…

# Différentes stratégies

- **Centralisée : Un serveur maître**
  - Arbitre d'un historique officiel
  - Tout ajout se fait par rapport à l'actualité du serveur
  - Simple, mais fragile et passe mal à l'échelle
- **Décentralisée : Des faits alternatifs**
  - Plusieurs historiques coexistent : moi, toi, le dépôt…
  - Des outils de synchronisation à grain plus fin

# Des historiques divergents

- **Branche = Un historique parmi d'autres**
  - Conséquence naturelle de la décentralisation
  - Peut être créé volontairement pour s'organiser
- **Quelques exemples d'utilisation**
  - Distinguer la production du développement
  - Représenter une version publiée sujette à correctifs
  - Isoler des développements simultanés

# Manipulation d'historique

- **Git permet de modifier l'historique, pour…**
  - Corriger une version incorrecte
  - Mettre à jour une branche sans *merge commits*
  - Rendre l'historique plus lisible pour autrui
- **Il faut le faire avec précaution**
  - Sinon, on risque des pertes de données, conflits…
  - Ces pratiques sont en général encadrées par le projet

# Forges sociales

- **Un ensemble cohérent d'outils web :**
  - Publication et visualisation de versions
  - Gestion de tickets (demandes, problèmes…)
  - Propositions de changements (+ analyse, discussions)
  - Hébergement : Paquets, site web, wiki, images Docker…
  - Notifications d'événements, statistiques, intégrations
- **Sur un serveur public, ou auto-hébergé**
  - Exemples : GitHub, GitLab, BitBucket/Stash…

# Plan de formation

- Nous explorerons git par « couches » conceptuelles



# TPs, mode d'emploi

- **Format : partie guidée + exercices + compléments**
  - Permet de s'adapter à vos rythmes variables
  - **Partie guidée = s'en tenir aux commandes indiquées !**
  - Vous pouvez créer un autre dépôt pour expérimenter
- **Pour tirer le maximum de cette formation…**
  - Offrez des vacances à vos téléphones / boîtes mail
  - N'hésitez pas à me solliciter, je suis là pour vous !

# Démo :
# VM, éditeur nano

# TP1 : Premiers pas